

Lab on Attacking the Lab

Can you open the shaders at your wish?

Pierre Ayoub
Giovanni Camurati
Aurélien Francillon

18/03/2022

About

Welcome to the lab session of the WiSec course! This lab is expected to span over the two WiSec lab sessions. Then it is quite long, so try to be efficient.

Today you will learn how to reverse engineer the radio protocol of the radio command to open and close the curtains of the lab room (and other rooms), and how to craft and transmit your own command packets. To warm up, you will learn how to listen to some radio communications with simple modulation schemes.

Be proactive, explore, and have fun!

Important Instructions

Make sure you read the following instructions carefully before going on.

Groups

Please do this lab in a group of two.

Answers Form

While following the instructions in this document, you will find some questions marked as **Question**, please answer them in [this form](#).

Important:

- Give a short and precise answer that shows that you understood it well. Do not copy paste from Web searches, you will not get any points. Do not waste time writing long answers.
- Save the link to your form so that you can access it again during the next sessions. If you are stuck at a question, fill the answer with dummy content and go on.

Setup Instructions

Hardware

In order to capture radio transmissions, a radio receiver is required – like an [RTL-SDR](#) dongle. If you do not have one, you can work with pre-recorded files, but you will be unable to do the initial exercises with real hardware (don't worry, no grade penalty).

PlutoSDR

In order to use this SDR (optional), you need a few more packages. On a Debian-based computer, run `sudo apt-get -y install libiio0 libiio-utils`. Connect it to your computer. Run the `iio_info -s` command, and note the USB URI (e.g. `[usb:1.14.5]`). Then, in GQRX, in *Device*, select *Other...* and in the *Device string*, use: `plutosdr,uri=usb:1.14.5`. If the command doesn't display any USB URI, you could try to use this device string: `plutosdr,uri=ip:pluto.local`.

Software

The lab requires [GQRX](#) and [Universal Radio Hacker](#) (URH). Those tools are installed on the lab computers.

If you prefer to use your own computer, on a Debian-based distribution, it will be probably easier to install the tools using the APT package manager and Pip:

```
sudo apt-get install gqrx-sdr ; pip3 install -U urh
```

You can also find and work with alternative tools, but we will not provide help on that.

Otherwise, we have prepared a virtual machine with the necessary tools already installed:

1. Download the [Virtual Machine in .ova](#).
2. Import the machine in [VirtualBox](#).
3. Username and password are **wisecslab** and **wisecslab**. Adjust resources, shared folders, and other settings if needed.

Warm Up: Listening to Some Interesting Audio

To warm up, you will listen to audio transmissions.

Familiarize yourself with the concept of Software Defined Radio (SDR) and have a look at the capabilities of the RTL-SDR dongle that you have.

Question 1: What is a Software Defined Radio (SDR)?

Question 2: What is the frequency range of the RTL-SDR?

Setup

You have a radio receiver (e.g., RTL-SDR)

Connect the antenna to the RTL-SDR dongle, and the dongle to the desktop. If you are using a virtual machine, make sure to add the USB device corresponding to the RTL-SDR in the list of shared devices in the VM settings, so that it is visible inside the VM. You can run `lsusb` in a terminal to check that it works. Run `gqrx -r` from a terminal and configure it for using the RTL-SDR dongle.

You do not have a radio receiver

If you do not have an SDR with you, you can play with a remote SDR via the web interface of a [WebSDR](#) (If this website doesn't work you can try to find another one [here](#) or [here](#)).

Listen to Local FM Radio Stations

Search online for the list of frequencies of local AM and FM radio stations, then listen to a few of them using gqrx or WebSDR.

Question 3: What is Automatic Gain Control (AGC)?

Question 4: What are the frequency and filter width parameters?

Question 5: Can you listen to an FM station with AM demodulation? If yes, how? If not, why?



Listen to Our Transmitter

You have a radio receiver (e.g., RTL-SDR)

Tune into frequency ~433.92 MHz to listen to our message and write it down.

You do not have a radio receiver

Download the [message.complex](#) file and open it in gqrx. To do so, use the IQ file option, and adjust the device string to:

```
file=/home/wisec/message.complex,freq=433.92e6,rate=192e3,repeat=true,throttle=true
```

Don't forget to set the path of the file and set *Input rate* accordingly. The audio message contains a secret message, try to recover it. The pre-shared key of the one-time pad is **wisec**. We used the [number_station_tx.py](#) script to generate the message to transmit, feel free to look at it.

Question 6: Which kind of radio station is it?

Question 7: Which modulation do we use?

Question 8: Which was the original secret message?

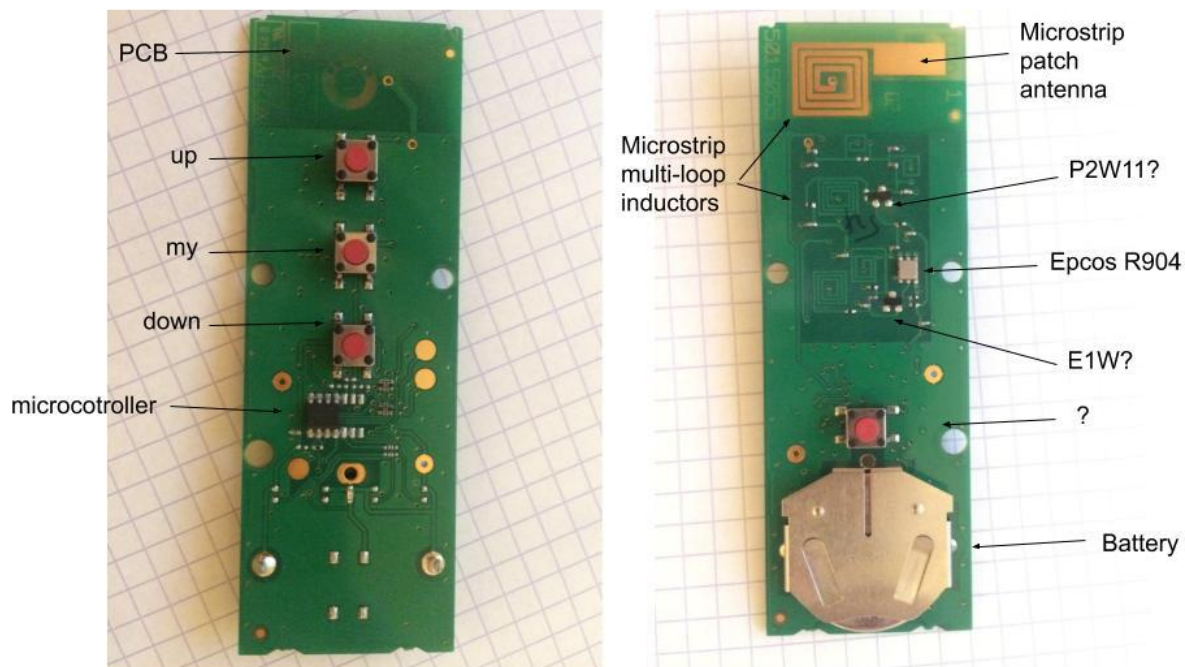
Question 9: How to activate the speaker only when there is a message?

Exercise: Reversing the Remote of the Lab's Curtains

In this part, you will understand how the radio protocol of the remotes for the curtains works. This is a necessary step to craft your own commands to the curtains.

Reconnaissance

Have a look at the remote control to see if you can get some useful information. We have already annotated some components for you in the picture below. The most important part is the resonator used to produce the carrier, as it tells us at which frequency we should look for the remote signal. Search online for the datasheet of that component and find the frequency of the carrier. For now, you do not need to look further into the hardware (there will be a section on this), but a first look at the PCB (Printed Circuit Board) may already give you some hints about the modulation scheme used by the remote controller.



Question 10: Why is there a resonator in the remote controller?

Question 11: Which is the frequency of the carrier?

Question 12: Any idea of plausible modulation schemes? Justify.

Using the Universal Radio Hacker (URH) to Reverse the Protocol

In this section, we will use a software, URH, to analyze the radio transmissions between the remote control and the curtains, in order to reverse the protocol.

You have a radio receiver (e.g., RTL-SDR)

Plug your RTL-SDR, open URH (type `urh` in a terminal), make sure everything is working. Be ready to launch the recording, call one of us and we will send some packets with the remote control, so that you can collect them.

You do not have a radio receiver

Download the [RTL-SDR-433_420MHz-1MSps.complex](#) file, which corresponds to pre-recorded samples. They correspond to 6 packets taken in the same lab with the same target address (correspond to the lab's curtains). The first 2 packets correspond to one button, the second 2 packets to another button, and the last two to yet another button.

Once you have some traces, the questions will guide you to figure out which modulation scheme, which encoding, which data protection are used, to finally decode some packets.

Important:

1. To save your URH settings, please create a project: *File, New Project...*
2. At load time, URH automatically tries to detect signal parameters. For some versions of URH and for some signals, it could lead to an overflow or another type of Python error where a maximum size is exceeded. To fix this error, disable this automatic analysis:
 1. Click on the "Edit" menu.
 2. Uncheck "Auto detect signals on loading" parameter.

1. Demodulation

We want to demodulate the signal we received. The first step is to identify the modulation scheme. To do so, in the *Interpretation* tab of URH, zoom and navigate into

the signal, and try to identify which physical parameter is used to carry information (carrier presence, amplitude/frequency/phase changes).

Question 13: Which modulation is used?

The second step is to find the symbol length. To do so, you will have to look for the smallest symbol. The best way is to select several symbols, look at the number of samples, and then divide by the number of symbols to have a more accurate result. Don't forget to take the sampling frequency into account when computing the length in μs .

Question 14: What is the symbol length in μs ?

Now that you have identified the physical parameters, by adjusting the symbol length, the tolerance and the padding settings, you will get a regular result for all messages inside the bottom window of URH, where the messages are displayed in bits after the demodulation process. From this, you should be able to distinguish between preamble messages, which are just short pulsations used to wake-up the receiver, and payload messages, the two composing one packet.

2. Decoding

Once we have demodulated our signal, we have a bunch of bits at our disposal, but it is highly probable that those bits are still encoded in a precise way and do not represent the actual data yet. The first step is to identify the encoding scheme that has been used in the payload messages, thus preamble messages could be ignored.

To do so, you will have to identify some patterns. Think about two things:

1. What fields can we expect to see? A synchronization field? A counter field that increments at every message?
2. Imagine how the encoding can encode those fields. Look at the bits two by two, search for noticeable patterns (like presence or absence of bits sequences), and make assumptions about the encoding scheme.

Question 15: Which kind of encoding does the remote use? What are the correspondences between received bits and logical bits?

3. Decryption

Once we have identified the encoding scheme, we will identify the different fields and decrypt the actual data. A methodology is to start with a very plausible field, like a counter, and look at its boundaries. Based on the knowledge that fields often contains an even number of bits, we can identify different fields at the right and the left of the first identified field, and identify dynamic and static fields across messages. You will use URH capabilities to discover the most fields as possible and make guesses about what they are. Here are some hints:

1. Switch between *Hex / Bin* display.
2. Use *Mark diffs in protocol*.
3. Label packet positions and try to identify possible components.
4. Look for known fields. Addresses and command types will be fixed, while counters and rolling codes will increase at each packet.

Question 16: Which fields do you expect to see? Do you recognize some of them?

Looking at the decoded data and the fields, you will see at the end of the messages a bunch of bits that are changing without any clear pattern between each message. You should recognize here that this is the result of a XOR operation. At this point, in real life, you would have to do some experiments with the packets to understand how the encryption/obfuscation of the message works. Here, we don't have enough time in the lab – so we give two links [\[1\]](#) [\[2\]](#) where the reverse engineering and the format of the protocol is given. From that, you should be able to understand all the fields and to write your own decoder in the next step.

URH is powerful enough to decode commonly used codes, but if there is more operation on the transmitter side, like cryptography or obfuscation, URH will need some help. Here, common decoders of URH will not be sufficient. You can resolve this by using a custom decoding script, which is a Python script – written by yourself – that will decode a packet and bring its output to URH by the standard output stream. Refer to URH's help to use your own custom script and how data is sent to it.

Download the [urh_decoder_template.py](#) file, which is the skeleton of the complete Python script -- we provide it for the sake of the lab's simplicity, but in real life, you would have to do everything yourself. Now, URH will send the demodulated and decoded data

to your script -- in the first argument, i.e. `sys.argv[1]` -- which has to decode it. Every decoding operation has to be implemented in Python, where it can be done in a lot of different ways. You can work in different bases (2, 10, 16), it's up to you to find which base is required for which decoding and decryption operation. Test them! To quickly test your script, you can either try it directly from the shell, or using the URH's *Test* field in the *Decoding* window, where you can enter a signal example and see the output in the *Decoded Bits*.

Question 17: What is the first packet once completely decoded and decrypted with the script? (Please, use our RTL-SDR-433_420MHz-1MSps.complex file provided above for this question).

Question 18: Does the XOR provide more security?

Question 19: Which attacks counter and rolling codes prevent? How?

The pages 1 to 3 of this formal document helps to understand how CRC works: [CRC Generating and Checking](#). You can also help yourself by looking at this informal article: [Cyclic Redundancy Check and Modulo-2 Division](#). In the previous blog posts, you can also find the code of the checksum algorithm used by this remote. This checksum algorithm corresponds to a CRC polynomial. In URH, by creating a “checksum” label, you can set a custom CRC polynomial for the checksum field which will be computed automatically, after the decoding (right-click on the label in the bottom window, then click on “Edit...”). Find the polynomial used by this protocol (look at Figure 1 of page 3 of the formal document), set it correctly in URH, set the correct Data range for the CRC computation, finally make sure that the CRC is correct for the decoded packets.

Question 20: What is a checksum? Which CRC polynomial is used here?

Question 21: Can you generate an "open" packet at any time? Can you do it if you have a recording of the last valid command?

Question 22: If you generate and send an “open” command, what will the legitimate user observe the next time?

Bonus: Spoof the Remote Control by Transmitting an "Open" Command

In this section, we want to spoof the user's legitimate command, since we have previously reverse engineered the protocol.

Write a custom encoder that does the exact opposite of your custom decoder, and select it in the Encoder field of the Decoding options. Try to configure the packet to send in the *Generator* tab of URH. First, remember the different fields you will need to modify to craft a valid packet. Finally, make sure every physical layer parameters are correct by clicking on the "Edit..." button.

Question 23: Concisely but precisely, describe all the different steps from the legitimate signal recording to the spoofed signal transmission. Dummy example: "1) Record signal at 90 MHz. 2) Change field Checksum from 10 to 11. [...]"

Congratulations, it is most likely that you are ready to open the curtains with your own packet. Call us, we will check your result. If everything is good, we let you collect a new packet and use an SDR to transmit your own.

Installing SDR Drivers

HackRF:

```
> cd /homes/$USER
> git clone https://github.com/mossmann/hackrf.git
> cd hackrf
> export PATH=$PATH:$PWD/host/build/hackrf-tools/src/
> urh # You can now use urh with the hackrf.
```

USRP B210:

```
> cd /homes/$USER
> wget http://files.ettus.com/binaries/images/uhd-images\_003.010.003.000-release.zip
> unzip uhd-images_003.010.003.000-release.zip

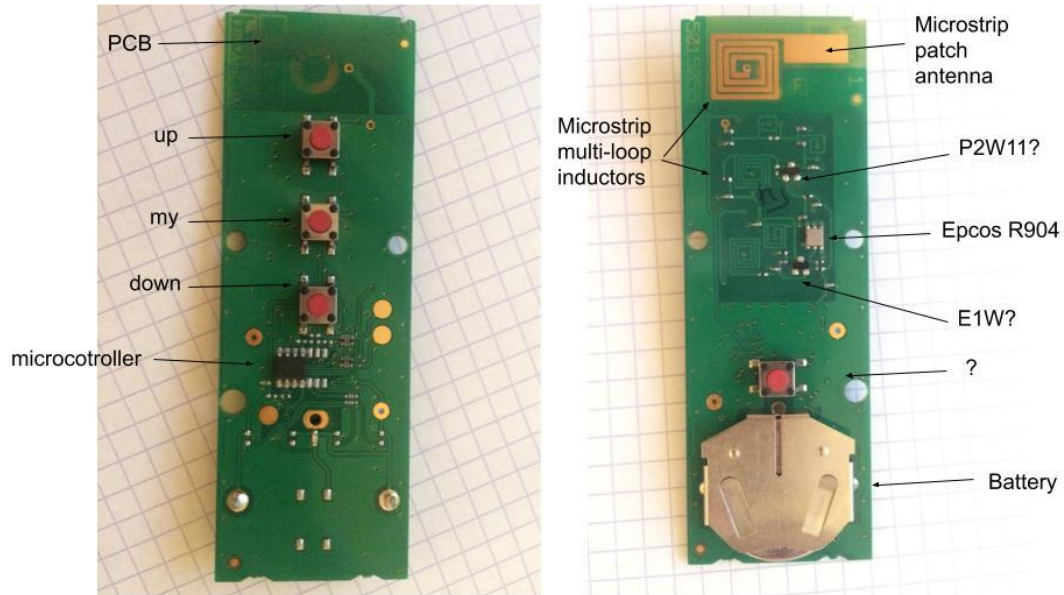
> imgs="/homes/$USER/uhd-images_003.010.003.000-release/share/uhd/images/"
> export UHD_IMAGES_DIR="$imgs"
```

```
> uhd_find_devices
```

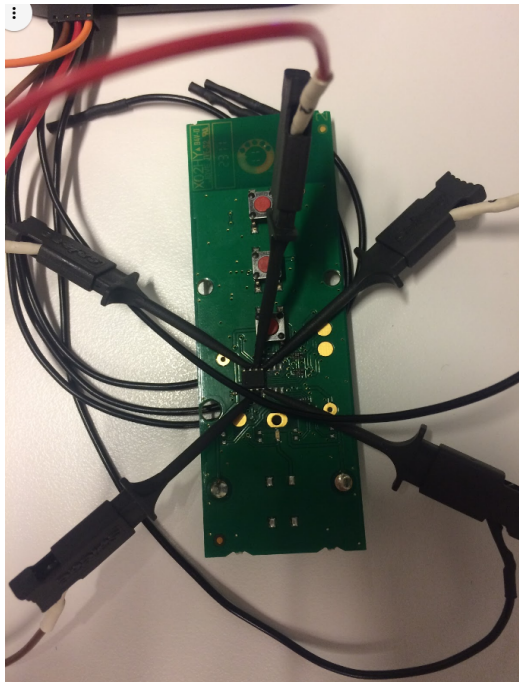
```
> urh
```

Bonus: Reversing the Hardware of the Remote

In this part, you will try to reverse the PCB to understand how it works. Here is an annotated picture:



We analyzed the signals on the PCB with a logic analyzer. The connections are visible in the image below:



You can find the traces collected with the logic analyzer from Salae [here](#). To visualize them, you need to download the software from [here](#). Execute the downloaded Applmage, then load the ".logicdata" file by clicking on *Options*, then *Open capture*.

Question 24: Explain what you understand about how this hardware implements the radio protocol you have reversed.